COMP3222, Machine Learning Technologies

Lab Sheet 5 (Weeks 6 and 7)

When: Weeks 6 and 7 (see the Moodle timetable).
Based on: Week 5 lectures, Classifier Evaluation and Classifier Comparison.

Aim. Compare classifiers on individual datasets and across multiple datasets. You will use stratified splits and cross-validation, calculate performance measures, tune classifiers, and interpret the results.

There are two tasks. Complete the core parts first; nested cross-validation, multiclass ROC curves and repeated multi-dataset experiments are extensions.

Code and data: use single_problem.py for Task 1 and bakeoff.py for Task 2 in the lab5 directory of the course repository.

https://github.com/TonyBagnall/COMP3222-2026

Task 1: Comparison on a single problem

Compare Random Forest, Rotation Forest, and a classifier you have implemented. Use the Wisconsin Diagnostic Breast Cancer dataset and Iris, available through sklearn.datasets.load_breast_cancer and load_iris. Treat each dataset as a separate experiment. If your own classifier cannot predict probabilities, use a library classifier for the probability-based measures.

1.1 Single train-test comparison

Use a stratified 70/30 train-test split with random_state=42. Use exactly the same split for all classifiers, and set their random_state where supported. Fit any preprocessing using only the training data, within a Pipeline where needed.

For both datasets, report accuracy, balanced accuracy, a labelled confusion matrix and log loss. For breast cancer, also report precision, recall, F1 and AUROC. For Iris, report macro-averaged precision, recall and F1, and macro-averaged one-versus-rest AUROC. See L5.1 slides 31–32 for the metric summary.

For breast cancer, treat malignant cases as positive. The scikit-learn loader codes malignant as 0 and benign as 1: recode malignant to 1 and benign to 0 before starting. Use the probability of this positive class for ROC/AUROC. Check the order of probability columns against classes_.

Save a labelled CSV results table. Save case identifiers, true labels, predictions and class probabilities so you can reproduce your analysis. Record the split seed and classifier settings.

1.2 Cross-validation comparison

Use 5-fold StratifiedKFold with shuffle=True and random_state=42, reusing the same folds for all classifiers. Collect out-of-fold predictions and probabilities: each case must be predicted by a model that was not fitted on that case. Compute the same measures as in 1.1 from the pooled out-of-fold results, including a confusion matrix.

Plot the three breast cancer ROC curves on one set of axes. Do classifier rankings change between metrics or between the single split and cross-validation? Which differences seem practically important?

Optional extension: plot one-versus-rest ROC curves for each Iris class. Make clear which class is positive in each plot.

1.3 Tuning

Reuse the train-test split from 1.1. Tune two classifiers with GridSearchCV using 5-fold stratified cross-validation on the training set only. Select parameters by accuracy. Keep any preprocessing inside the pipeline so it is fitted separately within each fold. The test set must not influence parameter selection.

Use a small, explicit grid. For Random Forest, for example, fix n_estimators=100 and try max_depth=[None, 5, 10] and min_samples_leaf=[1, 5]. Choose a similarly small grid for the second classifier.

Refit the selected configuration on the full training set and evaluate it on the untouched test set. Compare with the untuned results from 1.1 using the same measures. Report the selected parameters and fitting time. Does tuning improve every metric?

Optional extension: nested cross-validation

Reuse the outer folds from 1.2. Within each outer training fold, run a fresh GridSearchCV using, for example, 3 inner folds. Refit the selected configuration on that outer training fold, then predict its held-out fold. Aggregate the outer out-of-fold results and compare with 1.2. Do not tune once on the full dataset before cross-validation.

Task 2: Comparison on multiple problems

Use bakeoff.py and the continuous-feature datasets supplied in the repository, as in L5.2. Start with five datasets to check your code, then extend to the full supplied set for the comparison. Reuse your three classifiers and begin without tuning.

On each dataset, use a stratified 70/30 split with a fixed seed and identical train/test cases for all classifiers. Record accuracy, balanced accuracy and fitting time. Save results after each dataset so you can resume the experiment.

Build a results table with one row per dataset and one column per classifier for each measure. Report average ranks and pairwise win/draw/loss counts. Use aeon.visualisation.plot_pairwise_scatter to compare two classifiers, and plot_critical_difference for the overall comparison.

To match the Nemenyi approach in L5.2, explicitly set test="nemenyi" and alpha=0.05 in plot_critical_difference. Use lower_better=False for accuracy and balanced accuracy, or True for losses and fitting time. A non-significant difference does not establish equivalence.

aeon visualisation documentation

Optional extensions

Repeat the experiment using 10–30 stratified train-test resamples, with identical splits for all classifiers. Average each measure over resamples within each dataset before constructing the comparison table or running statistical tests. Do not treat resamples as independent datasets. If time permits, repeat with tuning performed independently within each training set.

For a comparison of two classifiers, apply a Wilcoxon signed-rank test to their paired dataset-level scores. Explain why applying the same test to overlapping resamples of a single dataset is problematic (L5.2 slide 18).

Show and discuss your results

Show a lab assistant your notebook or script, results table and plots. Explain which classifier you would choose, whether the metric changes that choice, and whether tuning helped. Save brief conclusions alongside your results. No separate report is required.